

The .cell Format: A Browser-Native Zero-Knowledge Document Protocol — Specification and Design Rationale

Enthropic Data LLC · Weddington, North Carolina, USA · 2026-10-09
(revision of 2026-08-09)

Published as a defensive disclosure. No patent is sought or asserted.

Author: David Brown · **Intended venue:** Technical Disclosure Commons (TDCcommons.org) · **License intent:** CC BY 4.0 (TDCcommons default) — maximum reuse is the objective

1. Abstract

Custodial document systems place decryption keys in the hands of the operator storing the data, which means the operator’s confidentiality guarantee is only as strong as its resistance to legal compulsion, breach, insider access, and service-layer deception — failure modes that require no broken cryptography, only continued custody of the keys. This disclosure describes the .cell file format and its reference cryptographic construction: content is encrypted client-side with AES-256-GCM; the content encryption key is wrapped per recipient via ephemeral ECDH P-256, HKDF-SHA256, and AES-KW, or split by Shamir Secret Sharing over GF(256) for threshold (quorum) access; advisory governance metadata — lifetime, distribution policy, quorum threshold, and lineage pointer — is bound to the ciphertext as AES-GCM additional authenticated data, computed over a deterministic canonical JSON serialization, so that the metadata is simultaneously plaintext-readable and cryptographically tamper-evident against any party lacking the content key; and every field is version-gated so that a document verifies under the rules declared by its own format version, permitting the specification to evolve without invalidating previously sealed documents. The server that stores and relays a .cell document holds ciphertext, wrapped keys, and public governance metadata only, and possesses no combination of stored material sufficient to recover plaintext. The disclosed mechanisms are independent of one another and may be adopted individually or in combination by any client-side or end-to-end encrypted document, messaging, or storage system.

2. Field of the Disclosure

This disclosure concerns client-side and end-to-end encryption of documents, specifically: the structure of an encrypted-document file envelope; authenticated-encryption constructions binding governance metadata to ciphertext; per-recipient and threshold (secret-sharing) key distribution; and format-versioning schemes that preserve backward verifiability across specification revisions. It specifies a complete `.cell` file format, the cryptographic primitives and parameter choices used to construct and verify it, and the security properties obtained, such that a reader skilled in applied cryptography and client-side web application development can construct an independent, interoperable implementation from this document alone. All cryptographic operations in the reference construction use only standardized, publicly specified primitives (AES-GCM, ECDH, HKDF, AES-KW, PBKDF2, ECDSA, SHA-256, and Shamir’s threshold scheme); no cryptographic primitive disclosed here is novel.

3. Background and Problem Addressed

A document protected only by transport encryption (TLS) and storage-layer encryption (“encryption at rest”) is protected against an outside attacker intercepting the network or physically obtaining storage media. It is not protected against the party that operates the storage service, because that party holds the decryption keys as an operational necessity — to render previews, index content for search, scan for malware, and recover accounts. Every confidentiality guarantee such a system offers reduces to a single question: what can compel, induce, or entitle the operator to disclose. The following failure modes each defeat a custodial system’s confidentiality guarantee without requiring any cryptographic primitive to be broken:

- **Legal compulsion.** A subpoena, warrant, or other legal process served on the custodian compels disclosure of any content the custodian can read; notice to the affected party is not guaranteed and is frequently prohibited by the same process. *Anchor: In re Lavabit, 2013 — an encrypted-email provider was ordered to surrender its TLS master key to satisfy a request targeting a single account, exposing the traffic of the entire user base.*
- **Breach.** Compromise of the custodian’s infrastructure exposes any content the custodian can read, regardless of the custodian’s intent or diligence. *Anchor: MOVEit Transfer, 2023 — a SQL-injection vulnerability in a managed file-transfer product enabled exfiltration of files from thousands of customer organizations.*

- **Insider access.** Personnel and contracted administrators with legitimate operational access to a custodial system are also positioned to exfiltrate or misuse any content that system can read, independent of any external attack.
- **Service-layer deception.** A custodian instructed to alter its own service behavior — rather than to disclose data it already possesses — can be made to capture credentials or plaintext as they pass through the custodian’s infrastructure in ordinary operation. *Anchor: Hushmail, 2007 — an “encrypted email” provider disclosed readable plaintext of specified users’ messages under court order by capturing passphrases at its servers during ordinary server-side operation, without breaking any cipher.*
- **Custody drift.** Acquisition, corporate restructuring, revision of terms of service, and bankruptcy proceedings each transfer effective custody of stored data to new parties and new incentive structures without any technical compromise occurring.

Each of these failure modes is available against any system architecture in which the party storing a document also holds, or can be made to hold, the means to decrypt it. Encryption in transit and encryption at rest do not address any of them, because in both cases the operator terminates the protection and is exempted from the threat model by construction.

4. Summary of the Disclosure

This disclosure describes a document file format, `.cell`, and its reference cryptographic construction, in which every property listed in §3 is addressed structurally rather than by policy: the operator storing a `.cell` document is removed from the set of parties capable of decrypting it, not asked to refrain from decrypting it. Content is encrypted entirely on the client before transmission; the server receives and stores only ciphertext, per-recipient wrapped key material, and metadata that is either inherently public (timestamps, format version) or deliberately kept both plaintext-readable and cryptographically tamper-evident. No server-side operation, credential, or administrative privilege — however extensive — is sufficient to recover plaintext from stored `.cell` data, because the server at no point possesses, generates, or has access to the content encryption key. §5 below is the complete normative specification of the format and its cryptographic construction; §6 sets out the specific design alternatives considered and rejected in reaching that construction, together with the properties obtained by the choices actually made; §7 enumerates the variants and substitutions the disclosure is intended to cover.

5. Detailed Description — Normative Specification

The following is the `.cell v1.3` format specification, reproduced in full from `docs/SPEC.md`. It is the normative enabling disclosure; §6, §7, and §8 of this document are commentary upon it and do not amend it.

Exactly what is anchored, and what is not. The timestamp anchors in §10 attest to the specification as it stood at repository tag `cell-format-v1.2-defensive-pub` (commit 62a8143), whose SHA-256 is 2e1ac039...bc7c6c41. **The text reproduced below is v1.3, which post-dates that anchor.** The two are separated deliberately, and a reader should hold them apart:

- **What the v1.2 anchor covers** is every mechanism common to both revisions — the envelope structure, the AAD binding, the canonical serialization, the per-recipient ECDH→HKDF→AES-KW fan-out, the Shamir quorum, the audit chain, and the version gating. These are unchanged in substance from the anchored bytes, and their priority date is 2026-07-18 as attested.
- **What it does not cover** is the v1.3 lifetime split of §8 — advisory versus disposal, and the rename of `expires_at` to `retain_until`. That is a new mechanism disclosed here for the first time, separately stamped on 2026-08-09; its priority dates from that stamp and from this publication, not from the 2026-07-18 anchor. §10 records both.

Corrections carried in the text below, not present in the anchored bytes. An accuracy review on 2026-08-09 checked every claim in the specification against the reference implementation and found errors in the anchored text. They are corrected below and enumerated in the 1.3 (corrections) changelog row. Two matter to anyone implementing from the anchored version:

- The anchored text specifies **base64url** for every binary field. This was wrong. The format has always used **standard base64**, and a decoder following the anchored text rejects real cells. Corrected in §4.0.
- The anchored text does not state the **ECDSA signature encoding**. It is the raw 64-byte `r || s` (IEEE P1363), not DER. Corrected in §4.1. This omission is the one gap that defeated an independent spec-only verification of a real cell.

Neither correction alters the format, and no cell ever written is affected: the implementation was always right and the document was wrong. They are recorded rather than quietly fixed because a disclosure that silently improved on its own anchored text would be asking the reader to trust exactly what it tells them not to.

Two further documentation revisions, also carried below (1.3 (corrections 2) and 1.3 (corrections 3) in §17, both 2026-08-23, neither altering the format). Corrections 2 fixes three places where the document contradicted itself: §15's worked example wrote "share_index": null against §6.4; §10 omitted v1.3 from the canonical-serialization set; and §14 omitted file:// from the secure contexts. Corrections 3 makes §4.1.1 self-contained. The canonical serialization had been defined by deferring to JavaScript's `JSON.stringify`, which silently imports four rules no other language's JSON encoder follows by default — number formatting, UTF-16 key order, the exact escape set, and (for v1.0/v1.1) key-order preservation on parse. Each of those rules, at some point, produced a non-matching header_hash while the format was being ported to Python, Go, Rust and Java. The text below is therefore docs/SPEC.md at digest 0412968e...0aad9e0b, separately anchored (§10).

One passage present in docs/SPEC.md is **omitted** below: a repository-internal note on licensing, whose links point to files inside the source repository. Its substance — that the specification is unrestricted and not covered by the licence applying to the repository's source code — is stated directly in §11 of this document, which governs for the purposes of this disclosure. No technical content is omitted.

Two purely presentational changes have been applied: section heading levels are demoted one level so the specification nests within this document, and several long code listings are re-wrapped to a narrower column. No token, parameter, identifier, or algorithm has been altered by either change. A reader wanting the exact anchored bytes should obtain them from docs/defensive-pub/anchors/ or at the tag, rather than transcribe them from this reproduction (§10).

Entropic Data LLC · entropicdata.com · Weddington, NC

Published as a defensive disclosure — freely implementable; no patent asserted.

Since v1.3: lifetime is **split by enforcer** (§8) — advisory, which conforming clients honour but which is *not* enforced against a keyholder, and disposal,

which the storing operator performs and which *is*. `expires_at` is renamed `retain_until`, because the old name asserted a wall the mechanism never built.

Since v1.1/v1.2: the advisory header metadata (threshold, lifetime, policy, lineage) is cryptographically bound to the ciphertext as AES-GCM *additional authenticated data* (§4.4), and all integrity hashes and signatures are computed over a **canonical JSON** serialization (§4.1.1) so that re-formatting the file (key order, whitespace) no longer breaks verification.

Earlier 1.0/1.1/1.2 cells remain openable unchanged (§12).

1. Overview

Cellular Defense (CD) is a browser-native, zero-knowledge encrypted document platform. Documents are encrypted entirely within the sender’s browser before leaving the device. The server stores only opaque ciphertext it cannot read under any circumstances. Recipients decrypt entirely within their own browser.

This document specifies the `.cell` file format, the cryptographic primitives, the access control architecture, and the security properties that can be independently verified. It is intended for security researchers, third-party implementers, and technical evaluators.

All cryptographic operations use the browser’s **Web Crypto API** (W3C standard). No third-party cryptographic libraries are used.

2. Zero-Knowledge Architecture

Every `.cell` document is protected by two independent cryptographic layers.

Layer	What It Protects	Algorithm	What an observer sees
1	Document content	AES-256-GCM	Ciphertext only
2	Content Encryption Key (CEK)	ECDH P-256 + HKDF + AES-KW	Wrapped CEK only

2.1 What the server/observer never holds

- ✗ Plaintext document content
- ✗ The Content Encryption Key (CEK)
- ✗ Any recipient private key
- ✗ Filename, MIME type, or file size

- ✗ Any key material sufficient to reconstruct a private key

2.2 What a `.cell` file exposes in plaintext

- ✓ Format version and document ID
- ✓ Cell creation timestamp
- ✓ Encrypted CEK shares (useless without the recipient’s private key)
- ✓ Recipient method, label, and key fingerprint (public metadata only)
- ✓ Lifetime and policy settings
- ✓ Header hash and optional sender signature

3. File Formats

Extension	Description
<code>.cell</code>	Plain JSON — human-readable, inspectable in any text editor
<code>.celz</code>	gzip-compressed <code>.cell</code> — same structure, ~30% smaller, binary
<code>.cdpub</code>	Public key export — share with others to receive encrypted cells
<code>.cdkey</code>	Full key export (private + public) — use on another device; store securely

`.celz` files are produced by gzip-compressing the UTF-8 encoded JSON of the `.cell` document. The decompressed content is identical to the `.cell` format.

4. Cell Envelope — JSON Structure

4.0 Binary field encoding

Every binary field in a `.cell` — `iv`, `ciphertext`, `eph_spki`, `hkdf_salt`, `ct`, `salt`, `credential_id`, `prf_salt`, `spki`, `pkcs8`, `prev_hash`, `payload_hash`, `header_hash`, `header_sig`, `header_sig_key` — is **standard base64** (RFC 4648 §4): the A–Z a–z 0–9 + / alphabet, with = padding.

It is **not** base64url. There are no - or _ characters and padding is not stripped. The reference implementation uses the browser’s `btoa()/atob()`, and `atob()` raises `InvalidCharacterError` on a - or _, so a base64url-encoded cell is not merely non-canonical — it fails to parse.

Erratum. Specification versions up to and including the anchored v1.2 text described these fields as “base64url” throughout. That was an error in the document, not in the format: no implementation has ever emitted base64url,

and cells produced from 2026-06-08 onward are all standard base64. Corrected in v1.3. Implementers working from the anchored v1.2 text ([docs/defensive-pub/anchors/SPEC.md](#)) should read “base64url” as “base64” wherever it appears.

Decoders SHOULD accept the base64url alphabet on input for robustness. Encoders MUST emit standard base64.

4.1 Top-level fields

```
{
  "version":      "1.3",
  "doc_id":       "<ULID>",
  "created_at":   <unix timestamp – seconds since epoch>,
  "header":       { ... },
  "header_hash":  "<base64 SHA-256 of canonical header
                  serialization>",
  "header_sig":   "<base64 ECDSA-SHA256 signature> | null",
  "header_sig_by": "<self-asserted fingerprint label of signing key> |
                  null",
  "header_sig_key": "<base64 SPKI of signing public key> | null",
  "payload":      { ... }
}
```

doc_id — ULID (Universally Unique Lexicographically Sortable Identifier). 26 characters, Crockford base32. First 10 characters encode 48-bit millisecond timestamp; last 16 characters are 80-bit cryptographic random. Lexicographically sortable by creation time.

header_hash — SHA-256 of the header serialization (§4.1.1). For v1.2 and v1.3 this is the **canonical** form; for v1.0/v1.1 it is `JSON.stringify(header)`. **Mandatory on every non-legacy cell**, and verified on open before any key material is touched (§10). An opener MUST reject a cell whose **header_hash** is absent rather than treat absence as licence to skip the check — otherwise deleting one field disables the header, ciphertext, and signature checks together. Note that **header_hash** is an **unkeyed** hash: it detects accidental corruption and benign re-formatting differences, but an attacker who edits the header can recompute it. Authenticity of the header against tampering comes from **header_sig** (sender identity) and the AES-GCM AAD binding (§4.4), not from **header_hash** alone.

header_sig — Optional ECDSA P-256 signature over the same header serialization bytes used for **header_hash** (§4.1.1). Provides sender authenticity. The signing key is the sender’s ECDH P-256 key re-imported with ECDSA usage (same P-256 scalar, different algorithm label). If null, the cell is unsigned.

Signature encoding. `header_sig` is the **raw 64-byte $r \parallel s$ concatenation** (IEEE P1363 / the Web Crypto ECDSA output format), base64-encoded — **not** the DER SEQUENCE `{ r, s }` that OpenSSL, Java, Go, and most general-purpose libraries produce and expect by default. An implementer using such a library must convert between the two forms. This is the single most likely point of failure when verifying a `.cell` outside a browser.

Signature absence is not detectable. An attacker can delete `header_sig`, `header_sig_key`, and `header_sig_by` from a signed cell; the result is byte-indistinguishable from a cell that was never signed, because the signature is produced *after* the AAD is fixed and so nothing binds its presence. A verifier that requires an authenticated sender **MUST** compare the verified signing-key fingerprint against an expected value obtained out of band, and **MUST NOT** treat “opened without error” as evidence of authorship.

Trust note. A valid `header_sig` only proves the holder of the private key matching `header_sig_key` produced it. `header_sig_by` is a self-asserted label and **MUST NOT** be trusted on its own. A verifier should anchor trust to the real fingerprint of `header_sig_key` (`SHA-256(SPKI)[16]`) and resolve it against its own known keys/contacts before attributing identity.

4.1.1 Canonical serialization (v1.2+)

For v1.2 and v1.3, `header_hash`, `header_sig`, and the AAD (§4.4) are computed over a **deterministic** serialization so that re-ordering object keys or pretty-printing the JSON does not change the bytes (and therefore does not produce a false “tampered/forged” failure). The canonicalization is:

```
canonicalize(v):
  if v is null or not an object:    return JSON.stringify(v)
  if v is an array:                 return "[" + join(",",
    canonicalize(e==undefined ? null : e) for e in v) + "]"
  otherwise (object):
    keys = sort(keys of v where v[k] != undefined)    //
    lexicographic
    return "{" + join(",", JSON.stringify(k) + ":" + canonicalize(v[k])
    for k in keys) + "}"
```

This mirrors `JSON.stringify`’s primitive encoding and string escaping; the only differences are recursively sorted keys and no insignificant whitespace. v1.0/v1.1 cells

continue to be verified with their original `JSON.stringify` serialization for backward compatibility (§12).

Deferring to `JSON.stringify` is exact but not self-contained, and the four rules it silently imports are where independent implementations diverge. They are stated here normatively, because each of them changes the hashed bytes and none of them is the default behaviour of a general-purpose JSON encoder outside JavaScript.

Numbers. JSON has no integer type: every number is an IEEE-754 binary64. A serializer **MUST** convert a value to binary64 *before* formatting it, and **MUST** then format it exactly as ECMA-262 6.1.6.1.20 (`Number::toString`, radix 10) specifies — the shortest decimal string that round-trips to the same binary64, with no trailing `.0`, positional form when the decimal exponent n satisfies $-6 < n \leq 21$ and exponential form otherwise, an explicit `+` on a non-negative exponent, and `-0` written as `0`. `NaN` and the infinities cannot appear, JSON having no literals for them.

Value	Canonical form	A common wrong answer
1.0	1	1.0
-0	0	-0
1e20	1000000000000000000000	1e+20, 1.0E20
1e21	1e+21	10000000000000000000000
1e-6	0.000001	1e-06
1e-7	1e-7	1e-07, 0.0000001
5e-324	5e-324	4.9e-324
9007199254740993	9007199254740992	9007199254740993

The last row is the rule’s practical edge: a language with arbitrary-precision integers must round the literal through binary64 and emit the rounded value, because that is what a JavaScript reader will do with the same bytes. Emitting the exact integer produces a serialization no conforming reader can reproduce.

Key ordering. Keys sort by **UTF-16 code unit**, which is what `Array.prototype.sort` gives JavaScript — not by Unicode code point, and not by UTF-8 byte. The three orders agree throughout the Basic Multilingual Plane and disagree above it: a character outside the BMP is a surrogate pair in UTF-16 (`U+D800–U+DFFF`) and therefore sorts **below** `U+E000–U+FFFF`, where code-point and UTF-8 orders both place it above.

String escaping. Escape exactly six characters plus the remaining C0 controls: " as \", \ as \\, U+0008 as \b, U+0009 as \t, U+000A as \n, U+000C as \f, U+000D as \r, and every other code point below U+0020 as \u00xx with lowercase hex. Everything from U+0020 upward is emitted literally as UTF-8 — **including /, <, >, &, U+2028 and U+2029**. Escaping any of those is a common default: Go’s encoding/json escapes <, >, & for HTML safety and U+2028/U+2029 even with HTML escaping disabled, and several encoders escape /. Every such escape changes the bytes and therefore the hash.

Key order must survive parsing — v1.0/v1.1 only. Because those versions serialize the header in its original insertion order (§12), an implementation whose JSON parser stores objects in an unordered or re-sorted map cannot reconstruct the bytes such a cell was signed over, and cannot verify it at all. Verifying v1.0/v1.1 therefore requires an order-preserving parse. v1.2+ sorts the keys itself and has no such requirement.

Conformance vectors. conformance/vectors/canonical.json carries input/expected pairs for all of the above, generated from the reference implementation. They need no cryptography and fail loudly, which makes them the cheapest first test for a new implementation: a canonicalization that disagrees produces a header_hash that never matches, with nothing in the failure to indicate why.

4.2 Header

```
"header": {
  "prev_hash": "<base64 header_hash of predecessor cell> | null",
  "threshold": { "required": <M>, "of_total": <N> },
  "lifetime": { ... },
  "policy": { ... },
  "access_map": [ ... ],
  "payload_hash": "<base64 SHA-256 of raw ciphertext bytes>"
}
```

prev_hash — Links this cell to a predecessor in a Mitosis chain (see §9). Null for original cells.

payload_hash — SHA-256 of the raw base64-decoded ciphertext bytes. Committed inside the header so it is covered by header_hash and header_sig. If the ciphertext is modified after sealing, this check fails on open.

threshold — Quorum requirement. required is M; of_total is N. For non-quorum cells: required = 1. See §6 for Shamir Secret Sharing.

4.3 Payload

```
"payload": {  
  "alg": "AES-256-GCM",  
  "encoding": "base64+gzip",  
  "iv": "<base64 12-byte IV>",  
  "ciphertext": "<base64 - AES-256-GCM encrypted content>"  
}
```

The ciphertext contains a manifest-prefixed gzip stream (see §5). The AES-GCM authentication tag is appended to the ciphertext by WebCrypto and verified automatically on decryption.

4.4 Additional Authenticated Data (AAD) — v1.1+

The advisory metadata that is fixed *before* encryption — `prev_hash`, `threshold`, `lifetime`, `policy` — is bound to the ciphertext as AES-GCM **additional authenticated data**. AES-GCM authenticates AAD without encrypting it: the recipient supplies the same AAD at decrypt time, and any mismatch fails the authentication tag.

AAD construction:

```
meta = [ header.prev_hash, header.threshold, header.lifetime,  
          header.policy ] // array, fixed order  
AAD  = UTF-8 bytes of:  
          canonicalize(meta)      for v1.2 and v1.3  
          JSON.stringify(meta)   for v1.1
```

`access_map` and `payload_hash` are **not** in the AAD because they do not exist yet when the body is encrypted (the access map wraps the freshly generated CEK; `payload_hash` is derived from the resulting ciphertext). They are instead covered by `header_hash/header_sig`.

Properties:

- **Not strippable.** The recipient's decrypt always supplies the current header's metadata as AAD; there is no separate field an attacker can remove to disable the check.
- **Not downgradeable.** Decrypting a v1.2 cell *without* the AAD (e.g. by editing version to 1.0) also fails the tag, since the ciphertext was produced *with* the AAD. An attacker cannot re-encrypt without the CEK.
- This binds metadata against third-party/storage/transit tampering. It does not (and cannot) prevent a legitimate recipient — who necessarily recovers the

CEK — from ignoring `lifetime/policy`; those remain advisory toward authorized holders (§8, §9).

Cells without a `version` (legacy v2.x) and v1.0 cells carry no AAD; they decrypt with empty AAD.

5. Encrypted Payload Structure

The plaintext fed into AES-256-GCM is:

```
[ 4 bytes LE uint32 – manifest length ][ manifest JSON bytes ][ file
bytes ]
```

This entire concatenation is gzip-compressed and then encrypted. The server and any observer see only ciphertext — filename, MIME type, and file size are invisible.

5.1 Manifest JSON

```
{
  "filename":      "document.pdf",
  "content_type":  "application/pdf",
  "size":          284921,
  "meta":          { "note": "Q3 escrow docs – call before opening",
                    "case": "2026-0417" }
}
```

The manifest is recovered after successful decryption by reading the 4-byte length prefix, parsing that many bytes as UTF-8 JSON, then taking the remaining bytes as the file content.

`meta` (added 2026-07-18) is an **optional, sender-defined JSON object** carried inside the ciphertext — a note to the recipient, a case number, reply instructions, or any application-defined structure. Because it lives inside the encrypted manifest, it enjoys the same confidentiality and integrity as the file content: the server and any observer never see it, and tampering fails GCM authentication. Readers that predate this field ignore it; it requires no format-version bump.

5.2 Encoding stack (encrypt path)

```
original file bytes
→ prepend [4-byte manifest len][manifest JSON]
→ gzip compress entire buffer
→ AES-256-GCM encrypt with random 96-bit IV and 256-bit CEK
→ base64 encode
→ stored in payload.ciphertext
```

5.3 Decoding stack (decrypt path)

```
payload.ciphertext
→ base64 decode
→ AES-256-GCM decrypt (CEK + IV) – auth tag verified automatically
→ gzip decompress
→ read 4-byte LE length prefix
→ parse manifest JSON (that many bytes)
→ remaining bytes = original file content
```

6. Access Map

Each entry in `header.access_map` holds a wrapped copy of the CEK (or a Shamir share of the CEK for quorum cells) for one recipient.

6.1 ECDH P-256 entry

```
{
  "method":      "ecdh-p256",
  "label":       "Alice",
  "fingerprint": "<16 hex chars – first 8 bytes of SHA-256(SPKI)>",
  "share_index": "<integer 1..N> | null",
  "wrapped_cek": {
    "eph_spki":  "<base64 – ephemeral sender public key, SPKI format>",
    "hkdf_salt": "<base64 – 32 random bytes>",
    "ct":        "<base64 – AES-KW wrapped CEK or CEK share>"
  }
}
```

CEK wrapping (per recipient):

```

ephemeral_keypair = ECDH P-256 (generated fresh per recipient)
shared_bits       = ECDH(ephemeral_privkey, recipient_pubkey) → 256
bits
hkdf_salt         = 32 random bytes
hkdf_key          = HKDF-SHA256(shared_bits, salt,
    info="cellular-defense-cek-wrap-v1", 256 bits)
wrapped_cek       = AES-KW(cek_or_share, hkdf_key)

```

CEK unwrapping (recipient):

```

shared_bits       = ECDH(recipient_privkey, eph_spki)
hkdf_key          = HKDF-SHA256(shared_bits, hkdf_salt,
    info="cellular-defense-cek-wrap-v1", 256 bits)
cek_or_share      = AES-KW-unwrap(ct, hkdf_key)

```

6.2 PBKDF2 (passphrase) entry

```

{
  "method":      "pbkdf2",
  "label":       "My passphrase",
  "fingerprint": "<16 hex chars – SHA-256 of PBKDF2 salt>",
  "share_index": <integer> | null,
  "wrapped_cek": {
    "salt":      "<base64 – 32 random bytes>",
    "iterations": 600000,
    "ct":        "<base64 – AES-KW wrapped CEK or CEK share>"
  }
}

```

Key derivation:

```

base_key  = PBKDF2-SHA256(passphrase, salt, 600000 iterations) → AES-KW
256-bit key
wrapped   = AES-KW(cek_or_share, base_key)

```

6.3 YubiKey PRF (FIDO2) entry

```
{
  "method":      "yubikey-prf",
  "label":       "My YubiKey",
  "fingerprint": "<16 hex chars – SHA-256 of credential_id>",
  "share_index": <integer> | null,
  "wrapped_cek": {
    "credential_id": "<base64 – WebAuthn credential ID>",
    "prf_salt":      "<base64 – 32 random bytes, the PRF eval input>",
    "ct":            "<base64 – AES-KW wrapped CEK or CEK share>"
  }
}
```

The PRF output (32 bytes) is used directly as the AES-KW wrapping key. Requires HTTPS or localhost, Chrome or Chromium, and a FIDO2 key with the hmac-secret extension (YubiKey 5 series, firmware 5.2+).

6.4 share_index

For quorum cells (`threshold.required > 1`), each access map entry holds one Shamir share. `share_index` is the x-coordinate of that share (1-indexed, matching the share's polynomial evaluation point).

For non-quorum cells (`threshold.required = 1`) each entry holds the full CEK and the reference implementation **omits the `share_index` key entirely** rather than writing `null`. Both forms are accepted on read. Note that the choice is not cosmetic: `access_map` is covered by `header_hash`, and the canonical serialization of §4.1.1 drops undefined but preserves an explicit `null`, so a cell written with `"share_index": null` has a different `header_hash` from an otherwise identical cell written without the key. Each remains internally consistent and verifies correctly; they are simply not byte-identical. Writers SHOULD omit the key.

7. Quorum — Shamir Secret Sharing

When `threshold.required > 1`, the CEK is split into N shares using Shamir Secret Sharing over GF(256) before wrapping. Any M shares reconstruct the CEK; fewer than M shares reveal nothing.

7.1 Field arithmetic — GF(2⁸)

Polynomial arithmetic over GF(2⁸) with irreducible polynomial $x^8 + x^4 + x^3 + x + 1$ (0x11b). Multiplication uses the Russian Peasant algorithm. Multiplicative inverse: $\text{gfInv}(x) = x^{254} \bmod \text{poly}$ (Fermat's little theorem in GF(2⁸)).

7.2 Share generation

For each byte position b of the CEK:

1. Build a degree- $(k-1)$ polynomial f over $\text{GF}(256)$ where $f(0) = \text{secret}[b]$ and coefficients $f[1] \dots f[k-1]$ are random.
2. Evaluate $f(i)$ for $i = 1 \dots N$. Each evaluation is one byte of share i .

7.3 Secret reconstruction (Lagrange interpolation at $x=0$)

Given M shares $(x_1, y_1) \dots (x_m, y_m)$, for each byte position:

$$\text{secret}[b] = \sum_i y_i \cdot \prod_{j \neq i} (x_j / (x_i \oplus x_j)) \quad (\text{all operations in GF}(256))$$

8. Lifetime Model — split by enforcer (v1.3)

`header.lifetime` describes a cell's intended lifetime. As of v1.3 it is split into two halves according to who enforces each, because they carry materially different guarantees and must not be read as one.

```
"lifetime": {
  "type": "<type string>",

  "advisory": {
    "retain_until": <unix timestamp> | null,
    "release_at": <unix timestamp> | null,
    "single_use": <boolean>,
    "minimum_atl": <integer>
  },

  "disposal": {
    "at": <unix timestamp> | null,
    "action": "none" | "delete" | "archive"
  }
}
```

Type	Description	Typical advisory.retain_until	disposal.action
permanent	No end date	null	none
flash	Single use, short-lived	now + 1 hour	delete
session	Hours to days	configurable	delete
clinical	Months (healthcare use)	configurable	archive

Type	Description	Typical advisory.retain_until	disposal.action
record	Years (compliance/ legal)	configurable	archive
timed_release	Locked until advisory.release_at	configurable	configurable

8.1 lifetime.advisory — conformance behaviour, NOT enforced

These values instruct conforming software. **They are not enforced against a recipient who holds a qualifying key, and the specification does not claim they are.** Such a recipient can modify their client, or write an independent opener from this document, and recover the plaintext irrespective of every value in this object. This is not an implementation weakness to be corrected in a later version; it is a property of any scheme that hands a party both the ciphertext and the means to decrypt it.

What the AAD binding of §4.4 provides here is narrower and worth stating exactly: **a third party cannot alter, strip, or downgrade these values undetected.** The recipient is therefore guaranteed to see the sender’s true instructions. Nothing obliges them to follow them. AES-GCM verifies byte equality, not meaning — supplying a `retain_until` in the past as AAD authenticates perfectly well.

Conforming clients **MUST** perform these checks before any cryptographic operation:

1. **Timed release lock:** if `type = "timed_release"` and `advisory.release_at > now` → reject with “Timed release — unlocks <date>”.
2. **Retention:** if `advisory.retain_until` is set and `< now` → reject with “Retention period ended <date>”.

minimum_atl — minimum **Access Trust Level** required of a recipient before the application should permit open: an integer tier reflecting how strongly the recipient’s identity was verified at enrollment (e.g. 1 = email-verified, 2 = mailed-credential, 3 = in-person). Advisory in the sense above; the cryptography does not evaluate it. Its real enforcement point is fan-out (§6): a certificate below the floor is never wrapped a key in the first place. *Naming note: earlier application code used the field name `minimum_etl` (“enrollment trust level”) for the same concept — `minimum_atl` as specified here is canonical for the `.cell` format.*

8.2 lifetime.disposal — operator-side, and genuinely enforced

`disposal` governs what the party **storing** the ciphertext does with it, and is the half of the lifetime model that is actually enforced. A sweep deletes or archives

the stored bytes on schedule, without the operator reading or being able to read anything. Against every party that has not already taken its own copy, this is real: the object ceases to be retrievable.

- `disposal.at` — when the stored ciphertext becomes eligible for disposal.
- `disposal.action` — `delete` destroys it; `archive` withdraws it from circulation while its existence remains attested; `none` retains it.

`disposal.at` is deliberately independent of `advisory.retain_until`. They frequently coincide, but they answer different questions and legitimately differ — a `record` cell may remain readable by its recipients indefinitely while the operator archives the stored copy after a statutory period.

Clients **MUST NOT** refuse to open a cell on the basis of `disposal`. A passed disposal date is not a statement about the recipient’s permission; it is a statement about the operator’s retention schedule.

8.3 Why the split

Until v1.2 both halves lived in one flat object (`expires_at`, `on_expiry`, ...) under a single “advisory” grade. That undersold `on_expiry`, which is enforced, and oversold `expires_at`, which is not — and the field name “expires” asserted a wall the mechanism never built. The rename to `retain_until` applies the same vocabulary discipline the format already applies to `copy_protection`, which *warns* and is forbidden from claiming to *prevent*.

v1.0–v1.2 cells retain the flat shape and are read under their own rules (§12).

9. Policy

`header.policy` carries distribution and presentation rules committed into the signed header.

```
"policy": {
  "copy_protection":  "none" | "standard" | "strict",
  "watermark_mode":   "none" | "basic" | "full",
  "created_on_origin": "<URL of creating application>",
  "origin_sig":        "<CD server signature> | null"
}
```

Field	Description
<code>copy_protection</code>	Redistribution intent. <code>strict</code> triggers a visible warning to the recipient.

Field	Description
<code>watermark_mode</code>	Forensic watermarking intent. <code>basic</code> = recipient identity; <code>full</code> = forensic embedding. Actual rendering is application-layer.
<code>created_on_origin</code>	<code>window.location.origin</code> of the application that created the cell.
<code>origin_sig</code>	Reserved for CD server countersignature. <code>null</code> in standalone browser deployments.

Policy is enforced by the application layer only; it is not a cryptographic access control mechanism. Since v1.1 it is bound to the ciphertext as AAD (§4.4), so its values are tamper-evident against third parties even though they are not enforced against an authorized recipient.

10. Audit Chain

The cell format provides a tamper-evident audit chain. Serialization below is the canonical form (§4.1.1) for v1.2 and v1.3, or `JSON.stringify` for v1.0/v1.1:

```

file bytes
├─ encrypted in payload.ciphertext (AES-256-GCM, with metadata bound
   as AAD — §4.4)
├─ payload_hash = SHA-256(raw ciphertext bytes)
│   └─ committed inside header (covered by header_hash)
│       └─ header_hash = SHA-256(serialize(header))
│           └─ header_sig = ECDSA-SHA256(serialize(header),
│               sender_privkey) [optional]

```

On open, the verifier checks in order:

0. Advisory lifetime gates (§8.1) — cheapest of all, and they need nothing but the clock.
1. `header_hash` is **present** (reject if absent, §4.1) and matches `SHA-256(serialize(cell.header))`. Benign reformatting is tolerated by canonicalization.
2. `payload_hash` (inside header) matches `SHA-256(base64decode(payload.ciphertext))`.
3. If `header_sig` present: ECDSA-SHA256 over `serialize(cell.header)` verifies against `header_sig_key`.
4. **On decrypt**, the AES-GCM tag verifies the ciphertext *and* the AAD-bound metadata (§4.4); a mismatch — including any edit to `threshold/lifetime/policy/prev_hash` that slipped past the unkeyed `header_hash` — fails here.

Ordering is normative. Steps 0–3 **MUST** complete before *any* key material is touched — before the access-map loop, before AES-KW unwrapping (itself a decryption), before a Shamir quorum is assembled, and before a passphrase entry runs its 600,000-iteration PBKDF2 derivation. An opener that unwraps first and verifies afterwards does expensive, interactive work on behalf of a cell it is about to reject, and performs a decryption under a header it has not authenticated. Step 4 is intrinsic to the body decryption and necessarily comes last.

What this does and does not cover. Together these detect tampering with the ciphertext, with the AAD-bound metadata, and — provided `header_hash` is present, which step 1 now enforces — with the rest of the header including `access_map`. They do **not** detect the *removal* of `header_sig` (§4.1): an attacker who strips the signature produces something indistinguishable from an unsigned cell. Sender authenticity must be established by checking the verified fingerprint against an expected value, not by the absence of an error.

11. Mitosis (Cell Lineage)

Mitosis is the operation of re-encrypting a cell to create a new version — changing recipients, rotating keys, or updating policy. The new cell references its predecessor via `header.prev_hash`.

```
cell1: prev_hash = null
  └─ cell2: prev_hash = cell1.header_hash
    └─ cell3: prev_hash = cell2.header_hash
```

`prev_hash` equals the `header_hash` of the immediately preceding cell in the chain. This creates an auditable chain of custody for the document’s access history.

`prev_hash` is included in the header and thus covered by `header_hash`, `header_sig`, and (since v1.1) the AES-GCM AAD (§4.4). Tampering with the lineage pointer is detected by the audit chain.

12. Format Versioning

The `version` field identifies the cell format version. The reference implementation supports exactly the versions listed in `SUPPORTED_VERSIONS = { "1.0", "1.1", "1.2", "1.3" }`. New cells are written as **1.3** (`CELL_FORMAT_VERSION`).

Version	Status	Notes
1.0	Supported (read)	Original format. <code>header_hash/header_sig</code> over <code>JSON.stringify(header)</code> ; no AAD.
1.1	Supported (read)	Adds AES-GCM AAD over <code>JSON.stringify([prev_hash, threshold, lifetime, policy])</code> . Hash/sig still over <code>JSON.stringify(header)</code> .
1.2	Supported (read)	Canonical serialization (§4.1.1) for <code>header_hash/header_sig/AAD</code> . Flat lifetime (<code>expires_at/on_expiry</code>).
1.3	Current (write)	<code>lifetime</code> split by enforcer into advisory and disposal; <code>expires_at</code> renamed <code>retain_until</code> (§8). Serialization and AAD rules unchanged from 1.2. This specification.

Open is version-gated: each cell is verified and AAD-bound using the serialization rules of *its own* version, so 1.0/1.1/1.2 cells continue to open unchanged, flat `lifetime` included.

Two sets govern that routing, and implementers should hold them as sets rather than as inline comparisons:

- **Canonical serialization** applies to { "1.2", "1.3" }. Anything else uses `JSON.stringify`.
- **AAD binding** applies to { "1.1", "1.2", "1.3" }. Anything else decrypts with empty AAD.

An implementation that adds a version to `SUPPORTED_VERSIONS` but leaves an inline `version === "1.2"` test elsewhere will silently drop canonical serialization, or decrypt new cells with no AAD, with no error raised at any point. The reference implementation keeps `CANONICAL_VERSIONS` and `AAD_VERSIONS` as explicit sets for exactly this reason.

Unknown version behavior: If `cell.version` is present but not in `SUPPORTED_VERSIONS`, the opener rejects the cell with a clear error: “Unsupported cell format version — update the app to open this cell.”

Legacy v2.0 detection: If `cell.version` is absent and `cell.cd_version` is present, the cell is treated as legacy format. Legacy cells use `recipients` instead of `header.access_map` and `encrypted_body` instead of `payload`. The wrapped CEK in a

legacy entry has no `hkdf_salt` field — the raw ECDH shared secret (first 32 bytes) is used directly as the AES-KW key without HKDF.

13. Key File Formats

13.1 `.cdpub` — Public key

Shared with others to receive encrypted cells. Contains no private key material.

```
{
  "cd_pubkey": "2.0",
  "label": "Alice",
  "fingerprint": "<16 hex chars>",
  "method": "ecdh-p256",
  "spki": "<base64 – SubjectPublicKeyInfo, P-256>"
}
```

13.2 `.cdkey` — Private key export

Contains the full key record including private key. Must be stored securely.

```
{
  "cd_key": "2.0",
  "keyId": "<UUID>",
  "label": "My Key",
  "method": "ecdh-p256",
  "fingerprint": "<16 hex chars>",
  "spki": "<base64 – public key, SPKI format>",
  "pkcs8": "<base64 – private key, PKCS#8 format>",
  "created_at": <unix timestamp>,
  "exported_at": <unix timestamp>
}
```

13.3 Key fingerprint

`fingerprint = toHex(SHA-256(SPki bytes)).slice(0, 16)`

16 lowercase hex characters (first 8 bytes of the SHA-256 of the SPKI-encoded public key). Used to match access map entries to available keys.

14. Cryptographic Primitive Reference

Operation	Algorithm	Parameters
Content encryption	AES-256-GCM	256-bit random CEK, 96-bit random IV, 128-bit auth tag
CEK wrapping (ECDH)	AES-KW	256-bit key derived via HKDF
ECDH key exchange	ECDH P-256	Ephemeral sender keypair per recipient
Key derivation	HKDF-SHA256	256-bit output, 32-byte random salt, info = "cellular-defense-cek-wrap-v1"
CEK wrapping (passphrase)	AES-KW	256-bit key via PBKDF2-SHA256, 600,000 iterations, 32-byte salt
CEK wrapping (YubiKey)	AES-KW	256-bit key = PRF output (hmac-secret extension, 32 bytes)
GF(256) Shamir SSS	Over GF(2 ⁸)	Irreducible poly 0x11b (AES polynomial)
Metadata binding	AES-256-GCM AAD	prev_hash/threshold/lifetime/policy bound to ciphertext (v1.1+)
Header signing	ECDSA P-256 + SHA-256	Signs canonical header bytes (§4.1.1) for v1.2+; <code>JSON.stringify(header)</code> for v1.0/v1.1. Raw 64-byte <code>r s</code> (IEEE P1363), not DER — see §4.1
Audit hashes	SHA-256	header_hash, payload_hash, fingerprints
Document ID	ULID	48-bit ms timestamp + 80-bit CSPRNG, Crockford base32

Operation	Algorithm	Parameters
Randomness	<code>crypto.getRandomValues()</code>	Browser CSPRNG — never <code>Math.random()</code>

All operations use the **Web Crypto API** (`crypto.subtle`). Requires a secure context — `https://`, `localhost`, or a `file://` document.

15. Complete Example

A minimal v1.3 cell encrypted for a single ECDH recipient, no quorum, permanent lifetime, unsigned. The `prev_hash`/`threshold`/`lifetime`/`policy` are additionally bound as AES-GCM AAD (§4.4), and `header_hash` is over the canonical serialization (§4.1.1):

```
{
  "version": "1.3",
  "doc_id": "01JXXXXXXXXXXXXXXXXXXXXXXXXX",
  "created_at": 1780531200,
  "header": {
    "prev_hash": null,
    "threshold": { "required": 1, "of_total": 1 },
    "lifetime": {
      "type": "permanent",
      "advisory": {
        "retain_until": null,
        "release_at": null,
        "single_use": false,
        "minimum_atl": 1
      },
    },
    "disposal": {
      "at": null,
      "action": "none"
    }
  },
  "policy": {
    "copy_protection": "standard",
    "watermark_mode": "none",
    "created_on_origin": "http://localhost:8080",
    "origin_sig": null
  },
  "access_map": [
    {
      "method": "ecdh-p256",
      "label": "Alice",
    }
  ]
}
```

```

    "fingerprint": "a1b2c3d4e5f6a7b8",
    "wrapped_cek": {
      "eph_spki": "<base64 – ephemeral public key>",
      "hkdf_salt": "<base64 – 32 random bytes>",
      "ct":        "<base64 – AES-KW wrapped 256-bit CEK>"
    }
  },
  "payload_hash": "<base64 – SHA-256 of raw ciphertext bytes>"
},
"header_hash":    "<base64 – SHA-256 of canonicalize(header)>",
"header_sig":     null,
"header_sig_by":  null,
"header_sig_key": null,
"payload": {
  "alg":          "AES-256-GCM",
  "encoding":     "base64+gzip",
  "iv":           "<base64 – 12-byte IV>",
  "ciphertext":  "<base64 – gzip([4-byte len][manifest JSON][file
                  bytes]) encrypted>"
}
}

```

16. Security Properties

Property	Mechanism
Confidentiality	AES-256-GCM — AEAD; IND-CCA2 secure under unique nonces
Authenticity (content)	AES-256-GCM auth tag — verified on decrypt
Authenticity (metadata)	AES-GCM AAD binds threshold/lifetime/policy/prev_hash — tamper-evident, not strippable/downgradeable (v1.1+)
Integrity (header)	SHA-256 header_hash, mandatory and checked before any key use (unkeyed — corruption/format check; authenticity from header_sig + AAD)
Integrity (ciphertext)	SHA-256 payload_hash committed in header
Sender authenticity	ECDSA P-256 header_sig — optional, per-cell; trust anchored to header_sig_key fingerprint, not the header_sig_by label

Property	Mechanism
Reformat resilience	Canonical serialization — key-reorder/whitespace does not break hash/sig/AAD (v1.2+)
Forward secrecy (per-cell)	Fresh ephemeral ECDH keypair per recipient per cell
Key isolation	Each recipient’s CEK copy uses independent ephemeral keypair
Metadata confidentiality	Filename, type, size encrypted inside payload (§5)
Threshold access	Shamir SSS — M shares required; fewer than M shares reveal nothing (information-theoretically)
Lineage integrity	prev_hash covered by header_hash and header_sig
Version safety	Unknown versions rejected with clear error
Disposal (lifetime.disposal)	Enforced — the operator’s sweep deletes or archives the stored ciphertext on schedule, without reading it. Acts on every party that has not already taken a copy (§8.2)
Retention (lifetime.advisory)	Not enforced against a keyholder. Tamper-evident only: the AAD binding guarantees the recipient sees the sender’s true values, and no third party can alter them. A recipient holding the CEK can disregard them entirely (§8.1)

17. Changelog

Version	Date	Summary
1.0	2026-06-08	Initial published spec. ECDH+HKDF+AES-KW CEK wrapping; encrypted manifest; PBKDF2 and YubiKey PRF access methods; Shamir SSS quorum; full lifetime model; policy; header_hash/payload_hash audit chain; ECDSA header_sig; ULID doc_id; Mitosis prev_hash chain.
1.1	2026-06-09	Bind advisory metadata (prev_hash/threshold/lifetime/policy) to the ciphertext as AES-

Version	Date	Summary
		GCM AAD (§4.4) — tamper-evident, not strippable/downgradeable. AAD over <code>JSON.stringify(...)</code> ; hash/sig still over <code>JSON.stringify(header)</code> .
1.2	2026-06-09	Canonical serialization (§4.1.1) for <code>header_hash</code> , <code>header_sig</code> , and AAD, so re-formatting (key order, whitespace) no longer breaks verification. Signature trust clarified to anchor on <code>header_sig_key</code> , not the <code>header_sig_by</code> label. v1.0/v1.1 cells remain openable.
1.2 (editorial)	2026-07-19	Define <code>minimum_atl</code> (Access Trust Level, §8) and declare it canonical over the legacy <code>minimum_etl</code> field name. No format or wire change.
1.3	2026-08-09	Split <code>lifetime</code> by enforcer into <code>advisory</code> (conformance behaviour, not enforced against a keyholder) and <code>disposal</code> (operator-side, enforced against anyone without a copy), and rename <code>expires_at</code> to <code>advisory.retain_until</code> (§8). Motivated by a demonstration that an independent opener written from this specification recovers plaintext from a cell whose expiry has passed: the old flat shape graded an enforced field and an unenforced one identically, and the name “expires” claimed a guarantee the format does not provide. Serialization, AAD construction and all cryptographic primitives are unchanged; 1.0/1.1/1.2 cells are read under their own rules. See docs/design/temporal-enforcement.md .
1.3 (corrections)	2026-08-09	Documentation corrections from an accuracy review that verified every claim against the reference implementation. No format or wire change; no cell ever written is affected. (a) Binary fields are standard base64 , not base64url as stated since v1.0 — the document was wrong, the implementation was always right, and following the old text

Version	Date	Summary
		produced unparseable cells (§4.0). (b) <code>header_sig</code> is raw 64-byte <code>r s</code> (IEEE P1363), not DER — previously unstated, and the one gap that defeated an independent spec-only verification (§4.1). (c) <code>header_hash</code> is mandatory and its absence is a rejection; previously an opener could treat a missing hash as licence to skip the header, ciphertext and signature checks together (§4.1, §10). (d) Verification order is now normative: integrity precedes all key use (§10). (e) Signature <i>removal</i> is documented as undetectable (§4.1, §10). (f) §16 said “N-1 shares reveal nothing” where it meant fewer than M — N-1 is a quorum’s worth. (g) <code>share_index</code> is omitted, not null, on non-quorum cells (§6.4). See <code>docs/REVIEW-2026-08-09.md</code>.
1.3 (corrections 2)	2026-08-23	Three documentation corrections, found by a line-by-line review of the specification against itself and against the exposition built on it. No format or wire change; no cell ever written is affected. (a) §15’s complete example wrote “<code>share_index</code>”: null on a non-quorum cell, contradicting §6.4 and correction (g) above — the 2026-08-09 correction reached the normative text and missed the example, so the one artifact an implementer is most likely to copy still demonstrated the discouraged form. Example corrected to omit the key. (b) §10 gave the canonical serialization as applying to “v1.2”, omitting v1.3, which §4.1.1 and §12 both place in <code>CANONICAL_VERSIONS</code>. (c) §14 gave the secure-context requirement as “HTTPS or localhost”, omitting <code>file://</code> — a legitimate secure context and the one the offline, air-gapped sealing path depends on.
1.3 (corrections 3)	2026-08-23	§4.1.1 made self-contained. No format or wire change; no cell ever written is affected, and the reference

Version	Date	Summary
		implementation is unmodified. The canonical serialization was defined by deferring to JavaScript’s <code>JSON.stringify</code>, which is exact but silently imports four rules that are not the default behaviour of a JSON encoder in any other language: binary64 number formatting per ECMA-262 <code>Number::toString</code>, UTF-16 code-unit key ordering, the precise escape set (notably that <code>/</code>, <code><</code>, <code>></code>, <code>&</code>, <code>U+2028</code> and <code>U+2029</code> are not escaped), and — for v1.0/v1.1 only — the requirement that a parser preserve object key order at all. Each is now stated normatively with a worked table, because each independently produced a <code>header_hash</code> that never matched during the Python, Go, Rust and Java ports, and the failure carries no indication of its cause. Machine-checkable vectors accompany them at <code>conformance/vectors/canonical.json</code>.

Cellular Defense · Entropic Data LLC · entropicdata.com

Reproduced text ends. The remainder of this document is original commentary and is not part of the normative specification above.

6. Design Rationale

The specification in §5 is the enabling disclosure. This section records, for each of its four load-bearing design decisions, the principal alternative considered, why that alternative was rejected, and the property obtained by the choice actually made — evidence that these alternatives were identified and deliberately set aside as of the priority date in §10, not merely unconsidered.

6.1 AAD-binding vs. a separate keyed MAC

Alternative considered. Bind the governance metadata (`prev_hash`, `threshold`, `lifetime`, `policy`) with a conventional keyed message authentication code — e.g.,

HMAC-SHA256 — computed over the metadata with a dedicated authentication key, stored or transmitted as a separate signature-like field.

Why rejected. A keyed MAC requires that every party who must verify the binding possess the MAC key. Distributing that key to each recipient reintroduces, in parallel, the exact per-recipient key-distribution problem (the “fan-out”) already solved for the content encryption key itself — doubling the machinery required to protect metadata that is no more sensitive than the content it accompanies, and creating a second key whose custody must independently be reasoned about. The set of parties who must be able to verify the binding is, by construction, identical to the set of parties who can obtain the CEK; introducing a second, separately-distributed key duplicates a solved problem rather than reusing it.

Property achieved. Using the CEK itself as the implicit verification key — via AES-GCM’s native associated-data mechanism rather than a separate MAC — means verification of the metadata binding is arithmetic performed *inside* the same decryption operation that recovers the content, with no additional key material, no additional wrapped-key entries, and no additional trust relationship. The construction holds unconditionally for anonymous or unsigned documents, because the binding rides on the CEK rather than on any sender identity or signing key.

6.2 Shamir quorum vs. a computational or server-mediated threshold scheme

Alternative considered. Implement M-of-N access approval as a server-mediated workflow (e.g., an approval record requiring M distinct account actions before the server releases a decryption capability), or as a threshold *decryption* scheme resting on computational hardness — threshold RSA decryption, threshold ElGamal, or a distributed-key-generation scheme in which M parties jointly perform a decryption operation without any party reconstructing a whole key — rather than information-theoretic secret sharing of the content encryption key.

Why rejected. A server-mediated approval workflow places a *decision* — not a cryptographic fact — between a request and the plaintext; the decision is state held on a server, and server-held state is precisely the custodial failure mode addressed in §3 (a single administrative action, `UPDATE approvals SET status='granted'` or its equivalent, converts the approval into a formality regardless of how many parties were nominally required). A computational threshold-decryption scheme is a legitimate alternative, and carries one genuine advantage this construction does not: because the parties jointly compute a decryption rather than reassembling a key, no single participant ever holds the whole content key. It was nonetheless not adopted,

for two reasons. First, its below-threshold secrecy is *computational* — conditional on the hardness of an underlying problem and on the limits of an adversary’s resources — where secret sharing’s is not. Second, no such scheme is available in the W3C Web Cryptography API, so adopting one would require shipping a third-party or WebAssembly cryptographic implementation into the trust path, which this project’s construction rules exclude. The corresponding cost of the choice actually made is disclosed in §8: the combining participant does momentarily hold the reconstructed key.

Property achieved. Shamir’s construction over $\text{GF}(256)$ yields an **information-theoretic** guarantee: any fewer than M shares are, without qualification, consistent with every possible secret value with equal likelihood, so there is no computation an adversary with unlimited resources and unlimited time can perform to gain any advantage — the information is not concealed from such an adversary, it is *absent*. This is a strictly stronger species of guarantee, for the specific below-threshold-collusion property, than the computational guarantee protecting the ciphertext itself, and it removes any single server, administrator, or subset of fewer than M keyholders as a point of compulsion, breach, or subpoena.

6.3 Per-Cell version-gating vs. global format migration

Alternative considered. On revising the specification (e.g., introducing the AAD-binding of §4.4 or the canonical serialization of §4.1.1), migrate all previously issued documents to be verified under the *current* rules, or require all deployed openers to apply the newest verification logic universally to every document regardless of the version under which it was sealed.

Why rejected. A document’s plaintext-adjacent verification rules are fixed at the moment of sealing, because the authenticated-encryption tag was computed over the exact bytes and exact AAD construction in force at that moment. Applying a newer rule — for instance, requiring AAD that did not exist when an earlier document was sealed — to an already-sealed document does not verify it under stricter rules; it fails it outright, indistinguishable from tampering, for every document issued before the rule existed. A format specification revised for long-lived documents (multi-year retention obligations are within its own lifetime taxonomy, §8) cannot silently invalidate documents sealed under its own earlier, equally valid rules.

Property achieved. Selecting verification rules by each document’s own declared format-version field, rather than by the verifying software’s currently preferred rules, allows the specification to add genuine new guarantees (such as AAD-binding) to newly sealed documents without applying — and thereby failing —

those same guarantees against documents that never carried them. The same per-document version selection is the basis for the complementary security property of §12: a document declaring a version the opener does not recognize is rejected outright rather than processed under a default or nearest-known rule set, closing the downgrade-via-unknown-version class of attack that has historically affected protocols which fall back to a default when a version marker is not understood.

6.4 Canonical JSON serialization for hash, signature, and AAD stability

Alternative considered. Compute `header_hash`, `header_sig`, and the AAD of §4.4 over the header’s serialization as produced by the implementing language’s native JSON serializer (e.g., insertion-ordered `JSON.stringify` output), as was done in the format’s initial (v1.0) release.

Why rejected. Standard JSON serialization does not guarantee byte-stable output for semantically identical data: two serializers, or the same serializer applied to an object whose keys were inserted in a different order, can produce different byte sequences for values that are equal in every respect a JSON consumer cares about. Any tool that parses and re-serializes a `.cell` document — a database that reorders object keys, a pretty-printer, a well-intentioned reformatting utility — changes the hash, signature, and AAD-derived bytes without altering a single semantic value, causing a byte-exact integrity check to report tampering against a document that was never altered in any way that matters. This was an actual defect in the format’s v1.0 release, corrected in v1.2 (v1.1 had introduced the AAD binding but still computed it over `JSON.stringify`).

Property achieved. A deterministic canonical serialization — recursive lexicographic key sorting, no insignificant whitespace, and primitive/string encoding otherwise identical to standard JSON serialization — ensures that the bytes fed to every hash, signature, and AAD computation are a function of the document’s *values* rather than of any particular serializer’s formatting choices, so that reformatting, key-reordering, or round-tripping through a foreign JSON tool no longer produces a spurious integrity failure. The algorithm is deliberately minimal (it fits in the ten-line specification of §4.1.1) precisely because a canonicalization scheme complex enough to be implemented divergently by independent parties is itself a known historical source of vulnerabilities (e.g., in XML digital signature canonicalization); keeping the algorithm small is treated here as a security property of the canonicalization choice, not merely an implementation convenience.

7. Variations and Alternatives

The variants enumerated below are disclosed as part of this publication and are intended to be covered as prior art alongside the specific construction of §5. Each is a substitution a person of ordinary skill in applied cryptography would arrive at without invention, given the specification in §5; they are recited explicitly so that no such substitution may later be claimed as a novel design-around of the disclosed arrangement. The disclosed subject matter is the *arrangement*, not any particular parameter choice, and the arrangement holds under every substitution listed.

7.1 Content encryption. Any authenticated encryption mode accepting associated data may substitute for AES-256-GCM, including AES-128/192-GCM, AES-GCM-SIV, ChaCha20-Poly1305, XChaCha20-Poly1305, AES-CCM, AES-OCB, AES-EAX, and Ascon. Where a mode with a nonce-misuse-resistant profile is chosen, the initialization-vector generation requirement relaxes accordingly. Initialization vectors of other lengths, and authentication tags of other lengths, are contemplated.

7.2 Content-key wrapping and key agreement. The per-recipient wrapping of §6.1 may substitute any key-encapsulation or key-agreement mechanism for ephemeral ECDH P-256, including: ECDH over P-384, P-521, Curve25519/X25519, Curve448/X448, or brainpool curves; static-ephemeral or static-static ECDH; RSA-OAEP or RSA-KEM; and post-quantum key encapsulation, expressly including ML-KEM (Kyber) at any parameter set, HQC, Classic McEliece, and hybrid constructions combining a classical and a post-quantum KEM whose outputs are concatenated or combined through a key-derivation function. A wrapped-key entry carrying a post-quantum or hybrid encapsulation is contemplated as an additional `method` value in the access map, with the access map’s heterogeneity permitting classical and post-quantum entries to coexist in a single document.

7.3 Key derivation and key wrapping. HKDF-SHA256 may be substituted by HKDF over any hash function, by the NIST SP 800-56C one-step or two-step KDFs, by KMAC, by TLS-style PRFs, or by any extract-then-expand construction; the `info/context` string may be any domain-separating value and the salt any length including zero. AES-KW may be substituted by AES-KWP, by AES-GCM used as a key-wrapping mode, by ChaCha20-Poly1305, or by any authenticated encryption applied to key material.

7.4 Passphrase-derived access. PBKDF2-SHA256 at 600,000 iterations may be substituted by any password-based or memory-hard key-derivation function, expressly including Argon2 (any variant, including Argon2id), scrypt, bcrypt,

balloon hashing, and PBKDF2 at any other iteration count or with any other underlying hash. The choice of a memory-hard function in place of PBKDF2 is explicitly contemplated and is a parameter substitution, not a distinct arrangement.

7.5 Hardware-authenticator access. The WebAuthn PRF / FIDO2 `hmac-secret` method of §6.3 may be substituted by, or supplemented with, any hardware-held key source, including PIV and other smartcards, TPM-sealed keys, secure enclaves and platform keystores, hardware security modules, OpenPGP cards, and passkeys. Deriving a keypair *from* an authenticator’s pseudorandom-function output — such that the authenticator constitutes the key rather than merely guarding it — is contemplated, as is using the output solely as a wrapping key.

7.6 Threshold and quorum schemes. Shamir secret sharing over $GF(2^8)$ may be substituted by secret sharing over any finite field or ring, including $GF(2^k)$ for other k , prime fields, and integer-modular constructions; by Blakley’s hyper-plane scheme; by Asmuth–Bloom or other Chinese-remainder-theorem schemes; by additive or replicated secret sharing; by verifiable secret sharing (Feldman, Pedersen) where the trusted-dealer assumption of §8 is to be removed; by proactive or refreshable secret sharing; and by weighted, hierarchical, or general monotone access structures in place of a flat M-of-N threshold. Splitting a key-wrapping key, a private key, or a share of a share, rather than the content encryption key directly, is contemplated. Threshold decryption schemes in which no participant reconstructs a whole key are contemplated as substitutes as discussed in §6.2.

7.7 Canonical serialization. The canonicalization of §4.1.1 may be substituted by any deterministic serialization, expressly including JCS (RFC 8785), canonical CBOR (RFC 8949 §4.2), DER and other ASN.1 distinguished encodings, canonical S-expressions, sorted-key tag-length-value encodings, fixed-offset binary layouts, and deterministic Protocol Buffers. Substituting a hash of the serialized bytes for the bytes themselves, at any point where serialized bytes are consumed, is contemplated.

7.8 The bound metadata set. The fixed-order array `[prev_hash, threshold, lifetime, policy]` supplied as associated data may be enlarged, reduced, reordered, or reframed as an object, a concatenation of length-prefixed fields, or a hash of any of these. Additional fields expressly contemplated for binding include recipient identifiers, jurisdiction and data-residency tags, classification and sensitivity labels, retention schedules, licence or usage terms, regulatory scope markers, audit-end-point references, and organization or workspace identifiers. The binding property holds for whatever field set is chosen, provided the field set is fixed before content encryption and reconstructed identically at decryption.

7.9 Integrity, signatures, and lineage. SHA-256 may be substituted by any cryptographic hash, including SHA-384/512, SHA-3, and BLAKE2/BLAKE3, at any truncation length for fingerprints. ECDSA P-256 signatures may be substituted by ECDSA over other curves, Ed25519/Ed448, RSA-PSS, or post-quantum signature schemes expressly including ML-DSA (Dilithium), SLH-DSA (SPHINCS+), and Falcon. The single-parent lineage pointer may be generalized to multiple parents, to a Merkle directed acyclic graph, to an append-only or transparency-log structure, or to an externally anchored chain; lineage pointers may be signed independently of the header.

7.10 Container, encoding, and transport. The JSON envelope may be substituted by CBOR, MessagePack, Protocol Buffers, ASN.1, or a bespoke binary layout, and may be embedded within an existing container format including PDF, ZIP, OOXML, Matroska, and email MIME structures. The gzip compression step may be substituted by any compression algorithm (including Brotli, zstd, LZMA) or omitted entirely; where an adversary can influence plaintext content, omitting compression is the contemplated mitigation. Base64 encoding may be substituted by any binary-to-text encoding or omitted for binary containers. ULID document identifiers may be substituted by UUIDv4/UUIDv7, KSUID, snowflake identifiers, or content-addressed digests.

7.11 Version gating. Version selection may be by an explicit version string or integer, by a cryptographic suite or ciphersuite identifier, by structural detection, or by an out-of-band profile, provided that a version outside the recognized set is rejected rather than processed under a default or nearest-known rule set. The rejection-on-unknown-version property, not the particular encoding of the version marker, is the disclosed element.

7.12 Deployment and payload class. The arrangement is independent of transport and of storage medium, and applies whether documents are held by a hosted service, a self-hosted server, peer-to-peer transfer, removable media, or a content-addressed store. It is not limited to documents: the same construction applies to encrypted messages, message threads, streams, database rows and columns, backups and archives, firmware and software update images, log records, telemetry, and any payload accompanied by governance metadata that must be readable before decryption and unforgeable by intermediaries. Execution in a web browser is not required; the arrangement applies equally to native applications, mobile applications, command-line tools, and server-side implementations, and the restriction to platform-supplied cryptography in §5 is a construction rule of this project rather than a limitation of the disclosed arrangement.

8. Scope Limitations

The following limitations are stated explicitly and are not incidental omissions.

Advisory properties are not enforced against an authorized recipient. The `lifetime.advisory` and `policy` fields of §8–§9 of the specification in §5 — a declared retention date, a single-use marking, a redistribution or watermarking intent — are cryptographically bound to the ciphertext (§4.4) such that no third party lacking the content encryption key can alter, strip, or downgrade them undetected. They are not, and cannot be, self-enforcing against a recipient who has already lawfully recovered the content encryption key: such a recipient’s software, or the recipient directly, may disregard a stated expiry, retain plaintext past a declared deadline, or redistribute decrypted content, because a party in possession of plaintext is beyond the reach of any cryptographic enforcement mechanism. This is not a defect particular to this construction; it is a limit inherent to any client-side-enforced policy scheme, applicable to digital rights management generally, and it is disclosed here explicitly rather than left for a reader to discover. The guarantee actually obtained is precise: such fields are *tamper-evident to everyone* and *guaranteed against every party except the authorized recipient*, never *self-enforcing against the recipient*.

No cryptographic primitive disclosed here is novel. Every algorithm specified in §5 — AES-256-GCM, ECDH over P-256, HKDF-SHA256, AES-KW, PBKDF2, ECDSA, SHA-256, and Shamir’s 1979 threshold scheme over GF(256) — is a standardized primitive with an established public history of analysis; none is proposed as a new cryptographic primitive by this disclosure, and no claim of cryptographic novelty is made or intended for any primitive in isolation. The subject matter of this disclosure is the *arrangement*: which fields are bound to which cryptographic operation, in what order, under what per-document versioning discipline, and with what metadata kept simultaneously plaintext-readable and tamper-evident.

Server-observable metadata is enumerated, not eliminated. As stated in §2.2 of the specification in §5, a `.cell` document’s format version, document identifier, creation timestamp, recipient methods and key fingerprints, and `lifetime/policy` field values remain plaintext-readable by the storing server and any network intermediary, by design — governance metadata must be readable before decryption to be actionable (e.g., an expired document must be rejected without any key ceremony). Aggregated across many documents, this metadata constitutes an access graph (who received what, when) that is not eliminated by this disclosure and should be treated as sensitive by any deployment for which traffic analysis is a concern.

The dealer in a Shamir split is trusted. The threshold scheme of §7 of the specification in §5 does not include verifiable secret sharing; a party splitting a content encryption key into shares is assumed to deal those shares consistently. An inconsistently dealt split produces a reconstruction failure detectable by the authenticated-encryption tag (no plaintext is ever released from an incorrect reconstruction), but the construction as disclosed does not attribute which share, if any, was dealt incorrectly. Verifiable secret sharing is enumerated in §7.6 as a contemplated substitution where this assumption must be removed.

Quorum reconstruction concentrates the key in one participant. Because the disclosed construction secret-shares the content encryption key rather than performing a distributed decryption, the participant who combines the shares necessarily reconstructs the whole key and holds it for the duration of the decryption. Below the threshold the key exists nowhere; at and above the threshold it exists, briefly, in one place. A malicious or compromised combining participant can therefore retain it. This is the disclosed cost of the choice recorded in §6.2, and the threshold-decryption alternatives enumerated in §7.6 are the contemplated substitutions for deployments where it is unacceptable.

9. Figures

The figures below are drawn as schematics rather than as screen captures, because a figure in a disclosure exists to serve enablement: it should help a reader construct an implementation, not show that one exists. Every value in them is real, and every one is traceable to a committed file. Figures 1 and 2 are taken from a signed 1-of-1 document (`docs/figure-data/fig-single.cell`), Figure 3 from a signed 2-of-3 quorum document with heterogeneous access methods — two ECDH recipients and one PBKDF2 escrow entry — (`fig-quorum.cell`), and Figure 4 from a two-link lineage chain (`fig-quorum.cell` → `fig-child.cell`). All three are regenerated by `tools/make-figure-cells.mjs` and are committed precisely so that a reader can recheck every value quoted here.

None of these values is illustrative or invented. Each was independently reproduced from the specification text in §5 alone — in a different language, using a general-purpose cryptographic library (OpenSSL via Python’s `cryptography`) and no code from this project — by recomputing `header_hash` over the canonical serialization, recomputing `payload_hash` over the raw ciphertext bytes, deriving the signing key fingerprint as `SHA-256(SPKI)[:16]`, and verifying the ECDSA P-256 `header_sig` over the canonical header bytes. All four checks pass on all three cells. The verifier is

committed as `tools/verify-cell.py`; a reader may run it against the committed cells and obtain the same result, or discard it and write their own from §5.

That exercise is the enablement claim of this disclosure discharged in miniature — and it is reported here with its one failure intact, because the failure is the more useful datum. On the first attempt the signature check did *not* pass: the specification did not state that `header_sig` is the raw 64-byte `r || s` of IEEE P1363 rather than the DER encoding that every general-purpose library expects by default. The specification was imprecise at exactly the point where an independent implementer would discover it, which is what such an exercise is for. §4.1 now states the encoding, and the check passes. A disclosure claiming enablement should be tested by the standard it sets, and should say what the test found.

The exercise has since been repeated at full scale. Four further implementations of the complete format — in Python, Go, Rust and Java, under `impl/` in the reference repository and licensed Apache-2.0 — now seal and open cells interoperably with the JavaScript reference. All five are checked against one shared corpus under `conformance/`: 27 captured cells — 14 that must open, spanning every version from the legacy layout through v1.3, ECDH, passphrase and quorum access, and signed cells; and 13 tamper, downgrade and policy cases that must be refused — plus 225 canonical-serialization vectors that need no cryptography at all. (Hardware-key access cannot be captured as a fixed vector and is not in the corpus.) A cell sealed by any one implementation opens in all the others. Building the ports found the same kind of failure the verifier found, at greater scale: §4.1.1 had defined the canonical serialization by deferring to `JSON.stringify`, and each of the four rules that deferral silently imports (number formatting, UTF-16 key order, the escape set, key-order preservation for v1.0/v1.1) produced a `header_hash` that never matched at some point during the ports. §4.1.1 now states all four normatively, which is the 1.3 (corrections 3) row of §17. The reference implementation was never wrong, and no cell is affected; the text simply had not said what it did.

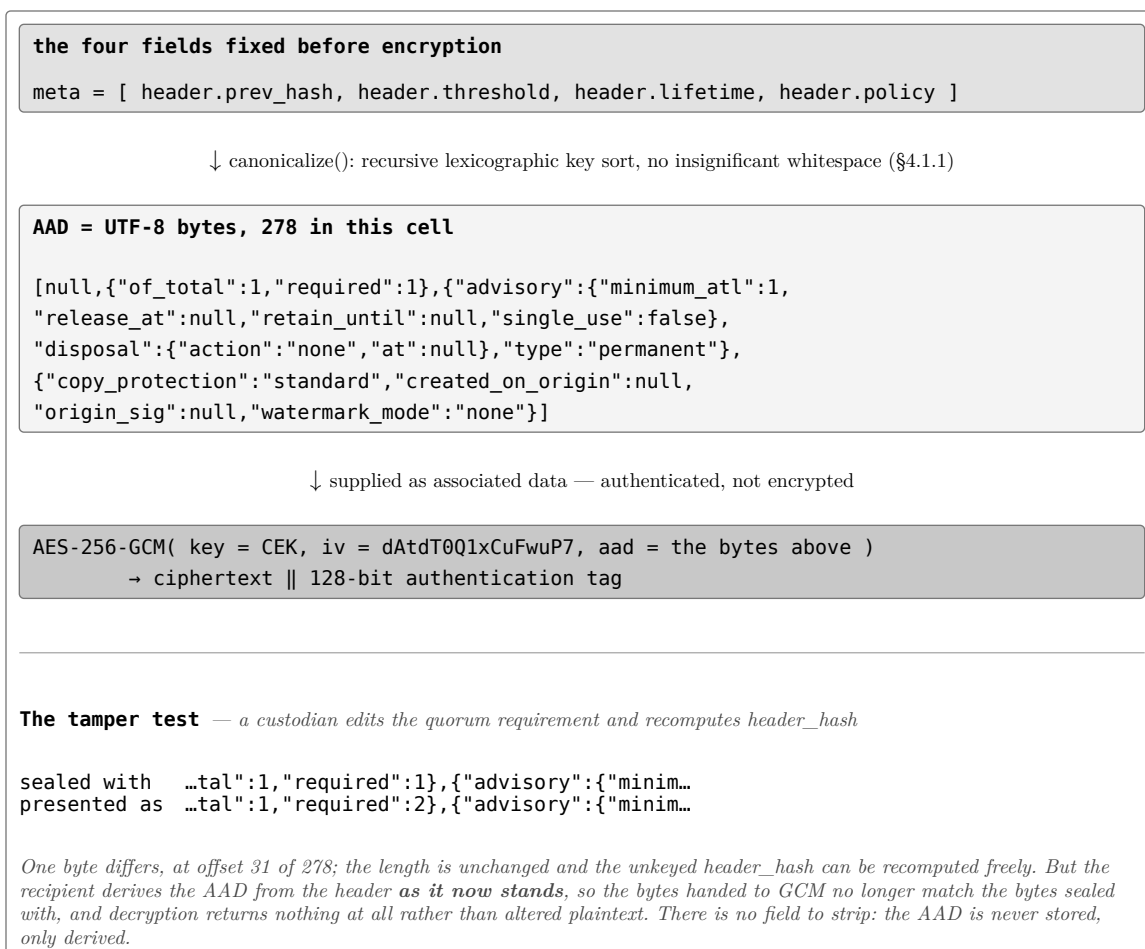


Figure 2: Binding advisory metadata to the ciphertext as AES-GCM additional authenticated data (§4.4), with the real 278-byte AAD of the cell in Figure 1. Below: altering the quorum requirement from 1 to 2 changes a single byte of that AAD and no other, and the authentication tag fails — the metadata is readable by anyone yet forgeable by no one.

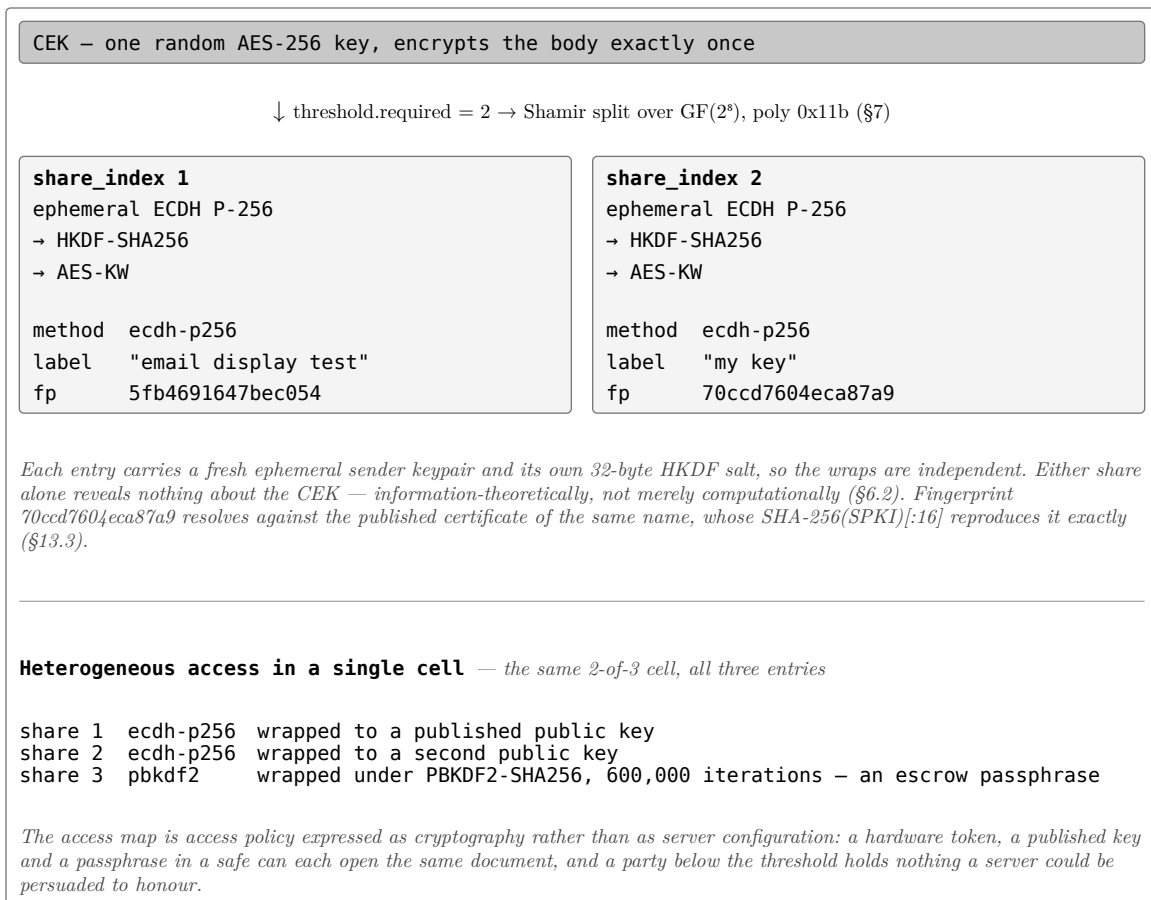


Figure 3: One ciphertext, one content key, N independently wrapped copies (§6). Upper: the real 2-of-3 quorum cell, where the CEK is Shamir-split before wrapping so no single holder can decrypt. Lower: the same machinery admitting heterogeneous access methods in one document.



Figure 4: The verification chain (§10) and lineage (§11), with real values from a two-link chain. Each layer commits to the next, so one optional signature at the top transitively vouches for everything beneath it.

10. Priority and Provenance

The design described in this disclosure was cryptographically timestamped, in the form of the pinned specification text reproduced in §5, before this document’s publication, using OpenTimestamps proofs anchored in the Bitcoin blockchain — an attestation dependent on no company, calendar server, or notary continuing to exist, independently and offline verifiable by any party.

Artifact	SHA-256	Anchored
docs/SPEC.md (as disclosed, tag cell-format-v1.2-defensive-pub)	2e1ac039 bfb1920b c5ba848f 2bfd904 e0c9ccfe 0443c016 034a6eb5 bc7c6c41	2026-07-18 · Bitcoin-confirmed 2026-07-19
cell-crypto.js (reference implementation)	11ce4b07 f2459ea1 b4677d98 e864a424 849afd70 80cafdfd ff60ca7b 81decae5	2026-07-18 · Bitcoin-confirmed 2026-07-19
docs/DEFENSIVE-PUBLISHING-GUIDE.md	6d97cc4e 048b6710 969b5116 ea774bc9 b63c5796 9aedc4eb 20d130f0 7dd95ee0	2026-07-18 · Bitcoin-confirmed 2026-07-19

Repository tag: cell-format-v1.2-defensive-pub (commit 62a8143) — in the private source repository. The anchored bytes themselves are published here under docs/defensive-pub/anchors/, and the digests above are what the proofs commit to; no access to that tag is required to verify them. **Bitcoin anchors:** block **958650** (merkle root cd57b1c2...3aead6, mined 2026-07-19 03:36:41 UTC) and block **958690** (merkle root d3619eda...a44858, mined 2026-07-19 07:14:39 UTC).

The v1.3 lifetime split of §8 is covered by a second, later anchor. It is a new mechanism and the 2026-07-18 anchor does not reach it; it was stamped separately on 2026-08-09 and confirmed the following day:

Artifact	SHA-256	Anchored
docs/SPEC.md (v1.3, tag cell-format-v1.3-defensive-pub)	870a5ea0 0846b5e0 be810adb 7e6c5e63 dbe0a7e1 d41a9ebc afdfba25 d8d20c0d	2026-08-09 · Bitcoin-confirmed 2026-08-10
cell-crypto.js (v1.3 reference implementation)	277cbcf e 784fb850 3c9b259a f5ffa871 9d521ab2 c482e859 5bbb1eef 8e7537bb	2026-08-09 · Bitcoin-confirmed 2026-08-10

Bitcoin anchors for v1.3: blocks **961803** (merkle root cba3701d...41f6, mined 2026-08-10 00:35:00 UTC), **961805** (7edb17cf...cb17, 00:41:25 UTC) and **961836** (f7128ba0...34bf, 04:42:43 UTC).

Every mechanism disclosed here is covered by one of those two anchors — the common core by 2026-07-18, the lifetime split by 2026-08-09 — with no part of the disclosure resting on publication date alone.

The exact text reproduced in §5 carries a third anchor. §5 is the v1.3 specification plus two later documentation revisions (see the note at the head of §5), which add no mechanism. That text was stamped as its own revision:

Artifact	SHA-256	Anchored
docs/SPEC.md (v1.3 + corrections 2 and 3)	0412968e ed2d6e16 831a8fa4 130eaf1a 9fedaad3 8fa3fceb 56aa12dd 0aad9e0b	2026-08-23 · Bitcoin- confirmed 2026-08-23/24

Bitcoin anchors for the corrected text: blocks **963756** (merkle root 264fb5ee...a95c, mined 2026-08-23 18:03:58 UTC), **963764** (daf695f1...bb9b, 20:32:26 UTC) and **963794** (d89b49d0...27f5, 2026-08-24 01:46:42 UTC). Hashing docs/defensive-pub/anchors/v1.3-corrections-3/SPEC.md reproduces the digest above.

To verify a digest yourself: the anchored bytes and their upgraded proofs are collected in **docs/defensive-pub/anchors/** — v1.2 at its root, v1.3 in anchors/v1.3/, the corrected text of §5 in anchors/v1.3-corrections-3/ — so no tag checkout is required. Hash the file with SHA-256, confirm it matches the value above, then run `ots info <file>.ots` and confirm it commits into the stated Bitcoin block, via your own node (`ots verify`) or any block explorer’s merkle root. Nothing in that procedure depends on this repository’s commit history: an OpenTimestamps proof commits to *file content*, so the digests can be checked against the files as published, and the signed tags recording the original revisions live in the private source repository from which this one was exported. This document, once submitted to a durable independent registry, becomes one further dated witness to the same design, corroborating rather than replacing the priority date already established by the anchors above.

One honest note on the proofs themselves. Between 2026-07-18 and 2026-08-09 the .ots files committed for docs/SPEC.md and cell-crypto.js were *pending-only stubs*: the Bitcoin attestations existed on the calendars but had never been folded into the committed proofs with `ots upgrade`. For those three weeks the repository asserted an anchor that a reader could not confirm from the committed files, even though the anchor was real. Every proof in docs/defensive-pub/anchors/ is now upgraded and carries its attestations. The same gap recurred once more: the proof for the corrected text of §5 was stamped on 2026-08-23 and not upgraded until 2026-10-08, although its blocks were mined within hours. The episode is recorded because the difference between *true* and *verifiable by a stranger* is the entire subject of this section, and a disclosure that quietly repaired such a gap would have failed its own standard.

A distinction the reader is owed: provable existence is not the same as public availability. The anchors above establish, beyond dispute, that the named bytes *existed* on the stated dates. They do not by themselves establish that those bytes were *publicly accessible* on those dates, and the two are separate requirements for prior art. The two therefore carry **different dates**, and this document asks to be read accordingly:

- **Existence** dates from the anchors — 2026-07-18 for v1.2, 2026-08-09 for v1.3, and 2026-08-23 for the corrected text of §5, each confirmed in the Bitcoin blocks listed above.
- **Public accessibility** dates from **2026-08-15**, when the reference repository was published at github.com/Entropic-Data-LLC/cd-vanilla-public, reachable anonymously and without an account. From that date the verification procedure described in the preceding paragraph can be carried out by any third party, and the specification text is retrievable by an interested member of the public without assistance.

Any assessment of this disclosure as prior art should treat the two requirements separately and should date accessibility from 2026-08-15 rather than from the anchors, notwithstanding that the anchored bytes and the published bytes are identical and can be shown to be so. That the earlier date cannot be claimed for accessibility is stated here rather than left to be discovered, because a disclosure that overstated its own reach would fail the standard it asks readers to hold it to.

11. Public Dedication

The `.cell` format specification, its cryptographic constructions, and every technical mechanism described in this disclosure are published as a defensive disclosure. No patent is sought or asserted over them; they are contributed to the public prior art and are freely implementable by anyone, for any purpose, without license or royalty. This dedication is intended to be permanent and irrevocable: no party, including Entropic Data LLC and the inventor named below, may later remove these techniques from the public's hands by seeking exclusive rights over them.

The reference *code* implementing this specification is a separate matter from the format disclosed here, and is licensed separately (the JavaScript reference implementation under the GNU Affero General Public License v3, with a commercial-license alternative available from Entropic Data LLC; the Python, Go, Rust and Java ports and the conformance suite under the Apache License 2.0); an implementation written independently from this specification is bound by none of that license and

owes its authors nothing. Where this disclosure and the normative specification it reproduces (§5) disagree with any other description of the format, the pinned specification text in §5, verified against §9, governs.

12. Inventor and Entity Information

Inventor: David Brown **Assignee / Defensive Publisher:** Enthropic Data LLC **Address:** Weddington, North Carolina, USA **Contact:** dbrown@entropicdata.com

Cellular Defense · Enthropic Data LLC · entropicdata.com — published as a defensive disclosure. No patent sought or asserted; freely implementable.